

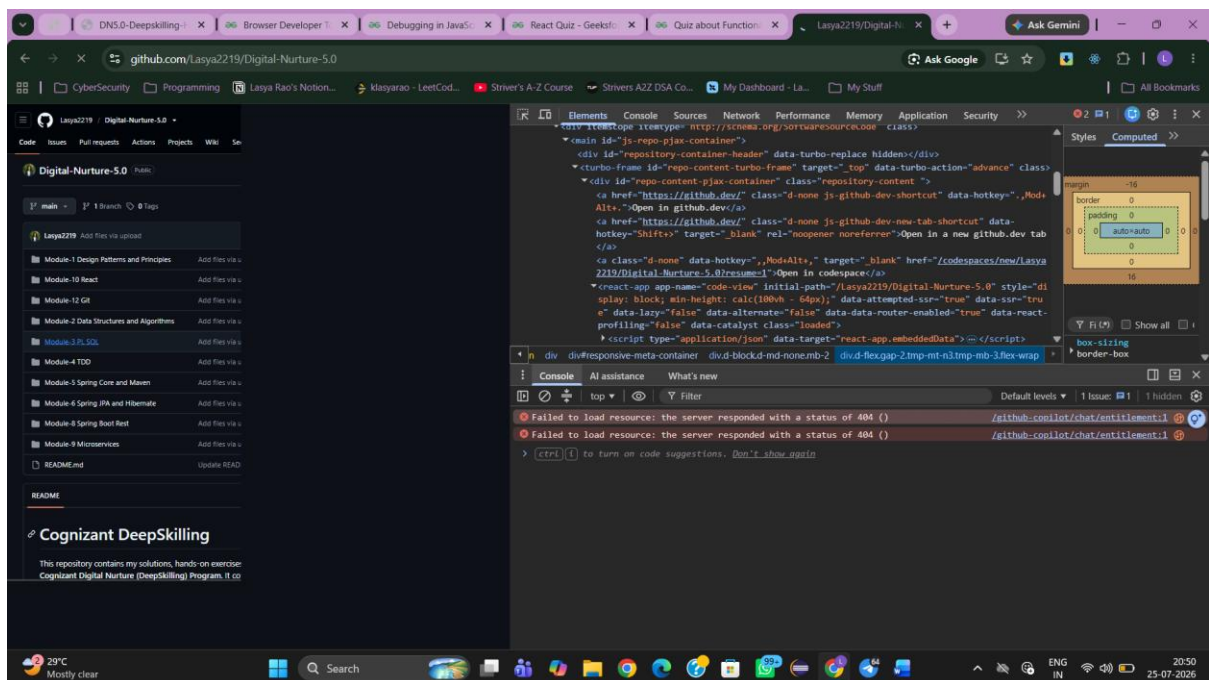
MODULE-11

APPLICATION DEBUGGING

BROWSER DEVELOPER TOOLS:

Tools to inspect, debug and optimise web pages through real-time interaction with HTML, CSS, Javascript, Network, Performance and accessibility features.

It is available in various web browsers (Chrome, Firefox, Edge etc).



To Access Browser Developer Tools: Right-Click -> Inspect

1. Inspector:

This panel displays and allows editing of the live DOM and applied CSS. Changes are reflected instantly but are not persisted after refresh.

Through Inspector you can:

- Inspecting HTML elements.
- Editing HTML and CSS in real time.
- Viewing applied styles.
- Checking the box model (margin, border, padding, content).

2. Console:

The Console is essential for JavaScript debugging. It logs errors, warnings, and messages from the page, and serves as a REPL (Read-Eval-Print Loop) for executing code live.

It is mainly used for:

- Viewing errors and warnings.
- Running JavaScript commands.
- Debugging application logic.
- Printing values using `console.log()`

3. Sources Panel:

This panel lets you inspect, debug, and edit JavaScript source files. Set breakpoints, step through code, watch variables, and inspect the call stack.

- Debugging JavaScript.
- Setting breakpoints.
- Stepping through code line by line.
- Inspecting variable values during execution.

4. Network Monitor:

Monitor all network requests and responses when loading or interacting with a page.

- Measuring page load performance.
- Identifying slow functions.
- Detecting rendering and scripting bottlenecks.
- Analysing CPU usage.

5. Performance Tools:

Record and analyse runtime performance, including JavaScript execution, layout, painting, and rendering.

- Record traces with detailed flame charts and timelines.
- Live Metrics and Insights sidebar (powered by Lighthouse data).
- Core Web Vitals (LCP, CLS, INP) monitoring with real-user comparison
- CPU/GPU throttling and mobile emulation.
- Identify bottlenecks in JS, CSS animations, and rendering.
- AI insights for recommendations.

6. Accessibility Inspector:

Inspects how a webpage is interpreted by assistive technologies to ensure accessibility.

- Viewing the accessibility tree.
- Checking roles, states, and properties.
- Testing colour contrast and keyboard navigation.
- Identifying accessibility issues.

DEBUGGING IN JAVASCRIPT

Types of Errors:

1. **Syntax Errors:** Issues with incorrect syntax, preventing execution.
2. **Runtime Errors:** Errors occurring during execution.
3. **Logical Errors:** Code runs without errors but produces incorrect results.

JavaScript Debuggers:

1. **Built-in Debugging Tools:** Accessible in Developer Tools. Console in Developer Tools in Chrome.
2. **console.log():** log messages to the console
3. **Setting Breakpoints:** Breakpoints stop execution at specific lines, allowing variable inspection. In Sources Tab of Developer Tools, click on the line numbers to set breakpoint
4. **debugger Keyword:** stops execution and opens debugging Tools.

Common Debugging Issues and Fixes:

1. **Debugging Syntax Errors:** Syntax errors occur when code violates JavaScript's syntax rules. These errors are typically caught during parsing before execution.
2. **console.log():** Logging variable values helps debug unexpected outputs.
3. **Breakpoints:** allow developers to pause code execution and inspect variable values.
4. **undefined Variables:** indicate scope issues or missing arguments.
5. **Catching Errors with try...catch:** Handling runtime errors prevents crashes. Useful for handling JSON parsing errors.
6. **Debugging Asynchronous Code with Promises:** Handling async operations can be tricky and .catch() handles rejected promises.
7. **Debugging Event Listeners:** Ensure correct event target is selected and ensure the element ID exists.
8. **Memory Leak Detection:** Memory leaks slow down application. Use Chrome DevTools to detect leaks.

DEBUGGING REACT APPLICATIONS

Use React Developer Tools to inspect React [components](#), edit [props](#) and [state](#), and identify performance problems.

Setting up:

1. Open React Developer Tools Chrome extension in the Web store.
2. Add to Chrome

Launching the Application:

1. Right-click -> inspect. On the same banner as *Elements*, *Sources*, *Console*, etc., there will be two new React tabs (only appear on sites using React):
 - Components
 - Profiler
2. Click on components tab
3. Click on gear icon to open settings and then click on Components tab in the pop-up window
4. Delete the filter to view hidden host (DOM) nodes.
5. Now, we will see a tree of App's contents. As we hover over the elements on the left, they are highlighted in the rendered view, similar to Chrome DevTools. If we click on the elements on the left side of the window, their properties are exposed on the right side.

Opening Chrome DevTools

- Open Chrome DevTools to inspect and debug the running application.
- Access tools for debugging HTML, CSS, JavaScript, and React.

Inspecting Elements

Examine and edit the webpage's HTML and CSS in real time. It is mainly used for:

- Viewing DOM elements.
- Editing HTML/CSS.
- Checking styles and layout.

Debugging JavaScript

You can find the JavaScript debugger in the Sources tab of [Chrome DevTools](#). Right-click the page and select Inspect to open the developer tools.

1. Using Breakpoint:

- Switch to the Sources tab and open the navigator sidebar. Here you can open the JavaScript file you want to debug and view its content.
- Once the code is displayed you can click on any line number to place a breakpoint.

2. 'debugger' statement:

- Add a debugger statement in your code editor.
- This also makes it easy to set a breakpoint when running code in the browser console.

Once execution is paused you can step through code line by line to see how data changes, or step into specific function calls to debug them in more detail.

Debugging React

React Developer Tools is a browser extension that adds two panels to Chrome DevTools: the Components panel and the Profiler panel.

These tools give deep visibility into React component tree, props, state, hooks, and rendering performance.

Components:

1. **Component Tree:** Browse the full component hierarchy. Click any component to inspect its props, state, and hooks.
2. **Search Components:** Search for components by name. Useful in large applications with hundreds of components.
3. **Edit Props/State:** Double-click any prop or state value to edit it live in the browser.
4. **Highlight Updates:** Toggle "Highlight updates when components render" to see which components re-render and how often.
5. **Profiler:** Record rendering performance and see exactly which components rendered, why, and how long each took.

1. Component Tab:

- Select any component in the Components panel to see its current state in the right sidebar.
- For function components with hooks, you will see each hook listed (State, Effect, Memo, Callback, Ref, Context) with its current value.

2. Profiler Tab:

The React Profiler records each "commit" (a batch of renders) and shows you exactly which components rendered and why. This is the definitive tool for finding unnecessary re-renders.

Steps:

- i. Open React DevTools > Profiler tab.
- ii. Click the record button (blue circle).
- iii. Perform the interaction you want to analyze.
- iv. Click stop. The flamegraph shows each commit.